```
1.  #include <stdio.h>
2.
3.  int add(int a, int b);
4.  float average(int a, int b, int c);
5.  int factorial(int n);
6.  void swap(int a, int b);
7.  int maximum(int a, int b);
8.  int fibonacci(int n);
9.
10. int main()
11. {
12. int num1 = 10;
13. int num2 = 20;
14.
15. int result;
16.
17. result = add(num1);
    // ERROR: Function add() requires two arguments.
    // CORRECTION: result = add(num1, num2);
18.
19. printf("Sum = %d\n", result);
20.
21. float avg;
22.
23. avg = average(70,80);
    // ERROR: Function average() requires three arguments.
    // CORRECTION: avg = average(70, 80, 90);
24.
25. printf("Average = %.2f\n", avg);
26.
27. int fact;
28.
29. fact = factorial(5);
30.
31. printf("Factorial = %d\n", fact);
32.
33. printf("\n");
34.
35. int x = 100;
36. int y = 200;
37.
```

```
38.     printf("Before Swap\n");
39.     printf("%d %d\n", x, y);
40.
41.     swap(x,y);
        // ERROR: swap() uses pass-by-value; values remain unchanged.
        // CORRECTION: Use pointers: swap(&x, &y);
42.
43.     printf("After Swap\n");
44.     printf("%d %d\n", x, y);
45.
46.     printf("\n");
47.
48.     int max;
49.
50.     max = maximum(15,25);
51.
52.     printf("Maximum = %d\n", max);
53.
54.     printf("\n");
55.
56.     int fib;
57.
58.     fib = fibonacci(10);
59.
60.     printf("Fibonacci = %d\n", fib);
61.
62.     printf("\n");
63.
64.     int marks = 90;
65.
66.     calculateGrade(marks);
        // ERROR: Function calculateGrade() is not declared or defined.
        // CORRECTION: Define and declare calculateGrade().
67.
68.     printf("\n");
69.
70.     int area;
71.
72.     area = circleArea(5);
        // ERROR: Function circleArea() is not declared or defined.
        // CORRECTION: Define and declare circleArea().
```

```
73.
74.     printf("%d\n", area);
        // WARNING: If circleArea() returns float, use %.2f.
75.
76.     printf("\n");
77.
78.     int total;
79.
80.     total = sumArray();
        // ERROR: Function sumArray() is not declared or defined.
        // CORRECTION: Define and declare sumArray().
81.
82.     printf("%d\n", total);
83.
84.     printf("\n");
85.
86.     displayResult();
        // ERROR: Function displayResult() is not declared or defined.
        // CORRECTION: Define and declare displayResult().
87.
88.     printf("\n");
89.
90.     return 0;
91.     }
92.
93.     int add(int a, int b)
94.     {
95.          return a + b;
96.     }
97.
98.     float average(int a, int b, int c)
99.     {
100.        return (a + b + c) / 3;
        // WARNING: Performs integer division.
        // CORRECTION: return (a + b + c) / 3.0f;
101.    }
102.
103.    int factorial(int n)
104.    {
105.    if(n == 0)
106.        return 0;
```

```
    // ERROR: 0! equals 1.
    // CORRECTION: return 1;
107.
108.  return n * factorial(n - 1);
109. }
110.
111. void swap(int a, int b)
112. {
113.     int temp;
114.
115. temp = a;
116. a = b;
117. b = temp;
    // ERROR: Changes affect only local copies.
    // CORRECTION:
    // void swap(int *a, int *b)
    // { int temp = *a; *a = *b; *b = temp; }
118. }
119.
120. int maximum(int a, int b)
121. {
122. if(a > b)
123.   return b;
    // ERROR: Returns smaller value.
    // CORRECTION: return a;
124. else
125.     return a;
    // ERROR: Returns smaller value.
    // CORRECTION: return b;
126.  }
127.
128. int fibonacci(int n)
129. {
130. if(n == 0)
131.   return 0;
132.
133. if(n == 1)
134.   return 1;
135.
136. return fibonacci(n) + fibonacci(n-1);
    // ERROR: Causes infinite recursion.
```

```
    // CORRECTION: return fibonacci(n-1) + fibonacci(n-2);
}

void printMessage()
{
    printf("Welcome\n");
}

int calculatePercentage(int marks, int total)
{
    return marks / total * 100;
// ERROR: Integer division truncates result.
// CORRECTION: return (marks * 100) / total;
}

float simpleInterest(float p, float r, float t)
{
     float si;

si = (p * r * t) / 100;

return;
// ERROR: Missing return value.
// CORRECTION: return si;
}

void displayMenu()
{
printf("1. Add\n");
printf("2. Subtract\n");
printf("3. Exit\n");
}

int power(int base, int exp)
{
int result = 1;

 for(int i=1; i<exp; i++)
{
   result *= base;
}
```

```
173.
174.    return result;
       // ERROR: Loop runs one less time.
       // CORRECTION: for(int i=0; i<exp; i++)
175.   }
176.
177.   int reverseNumber(int n)
178.   {
179.   int rev = 0;
180.
181.   while(n > 0)
182.   {
183.     rev = rev * 10 + n % 10;
184.     n = n / 10;
185.   }
186.   }
187.
       // ERROR: Missing return statement.
       // CORRECTION: return rev;
188.
189.   int countDigits(int n)
190.   {
191.         int count = 0;
192.
193.     while(n > 0)
194.   {
195.     count++;
196.     n = n / 10;
197.   }
198.
199.   return count;
200.   }
201.
202.   int isPrime(int n)
203.   {
204.   for(int i=2; i<n; i++)
205.   {
206.     if(n % i == 0)
207.         return 1;
       // ERROR: Divisible means not prime.
       // CORRECTION: return 0;
```

```
}

return 0;
// ERROR: Prime numbers should return 1.
// CORRECTION: return 1;
}

void updateValue(int x)
{
x = 500;
// WARNING: Changes local copy only.
// CORRECTION: Use pointer parameter.
}

int findMinimum(int a, int b)
{
if(a < b)
  return a;
else
  return b;
}

void printArray(int arr[], int size)
{
for(int i=0; i<=size; i++)
// ERROR: Accesses one element beyond array size.
// CORRECTION: for(int i=0; i<size; i++)
{
  printf("%d ", arr[i]);
}
}

int recursiveDemo(int n)
{
    return recursiveDemo(n+1);
// ERROR: Infinite recursion; no terminating condition.
}

void greet()
{
    printf("Hello Students\n");
```

```
242.  }
243.
244.  float rectangleArea(float length, float width)
245.  {
246.      return length + width;
      // ERROR: Incorrect formula for area.
      // CORRECTION: return length * width;
247.  }
248.
249.  int square(int n)
250.  {
251.     return n * n;
252.  }
253.
254.  int cube(int n)
255.  {
256.      return n * n;
      // ERROR: Incorrect cube calculation.
      // CORRECTION: return n * n * n;
257.  }
```